OGC2026 기술 보고서 — 조선소 블록 배치 문제

팀 MSGA · 초안 (2026-08-03) · 최종 제출 2026-08-06

> 작성 메모: 최종 PDF는 `[OGC2026_TechReport]_MSGA.pdf`, 코드 번들은 `[OGC2026_AlgorithmCode]_MSGA.zip`. 본 문서의 모든 수치는 저장소 내 측정 로그·소스 주석에서 인용했으며 출처를 병기한다.

---

1. 문제와 목적함수 분해

각 블록 i는 릴리스 R_i, 처리시간 P_i, 납기 D_i, 작업량 L_i, 베이 선호도 S_{ij}(합 100)를 가지며, K_i개의 다각형 레이어로 이루어진 3차원 구조물이다. 모든 블록에 대해 (베이, 방향, 위치, 진입시각)을 동시에 결정해야 하고, 목적은 $w_1 Z_1 + w_2 Z_2 + w_3 Z_3$의 최소화다. Z_1은 총 지연 $\Sigma_i \max(0, EXIT_i - D_i)$, Z_2는 베이 면적으로 정규화한 작업량 불균형의 최댓값, Z_3은 선호 후회 $\Sigma_i (S_i\max - S_{i,j(i)})$ 이다.

공개 예선 인스턴스에서 우리 해의 목적함수를 성분별로 분해하면 문제가 두 개의 서로 다른 게임임이 드러난다 (`docs/PIPELINE.md`, v31d4 보드):

	$w_1 Z_1$	$w_2 Z_2$	$w_3 Z_3$
P3	0	16,580 (19%)	69,150 (81%)
P4	2,226,611 (76%)	26,810	681,750 (23%)
P5	9,759,756 (93.5%)	18,508	661,143 (6.3%)
P6	31,474,507 (96.7%)	15,024	1,071,300 (3.3%)

P4–P6은 사실상 순수 지연 문제이고 P3은 순수 선호-후회 문제다. 이 분기는 하드코딩하지 않고 탐색 중 실시간으로 판정한다. `alns::run`은 매 평가마다 `z3dom` ($w_3 Z_3 > w_1 Z_1$)과 `prefStrong` ($Z_1 \le 0$ 이고 $Z_3 < n$)을 갱신하며, 이 두 무차원 술어가 파괴 연산자의 서식지 게이트와 삽입 시 베이 정렬 확률(0.70 → 0.92)을 바꾼다. 튜닝 상수가 아니라 인스턴스 자신의 가중치 비교이므로 히든 인스턴스에서도 의미가 보존된다.

구조적 사실 하나를 명시해 둔다: **배정이 고정되면 Z_2·Z_3은 상수, Z_1은 블록별 합이며 체커 규칙은 전부 베이 내부에서 닫힌다.** 이것이 가격화된 축출 연산자의 게이팅과 베이-국소 이동의 정당성 근거다(불변식 6).

2. 해 표현

블록 하나의 상태는 (베이 j, 방향 o, 정수 x, 정수 y, 진입 e)이며 퇴출은 **$f = e + P_i$ 항등식으로 유지**된다 (`core::Sol`). 즉 탐색 공간에서 일정은 e만의 함수이고 정시 조건은 $e \le D_i - P_i$로 단순화된다. 여러 연산자가 목표로 삼는 "정시 진입" $e^* = \max(R_i, D_i - P_i)$가 여기서 나온다. f를 더 늘리는 것은 규칙상 허용되지만 탐색 중에는 하지 않고, 체커 게이트의 구제 사다리에서만 최후 수단으로 쓴다.

출력은 일자별 연산 목록으로 렌더링한다. 같은 날의 EXIT는 모두 ENTRY보다 앞서고, **동시 EXIT는 나중에 진입한 블록부터** 배출한다(LIFO). 오라클의 규칙 3·5가 정확히 이 순서를 가정하므로 한쪽만 바꾸면 오라클이 체커와 갈라진다(불변식 7).

부수 성질 하나가 중요하다: **위치 (x,y)는 목적함수에 전혀 들어가지 않는다.** 따라서 잔류 블록을 밀어 모으는 공간 압축은 점수 중립이며 순수한 "여유 공간 생성" 연산이 된다(§5).

3. 실행가능성 오라클

`core::vpConservative(pb, s, A)` 는 블록 A 관점에서 체커의 다섯 규칙을 자기 베이 잔류 블록들에 대해 평가한다: (1) 시간이 겹치는 잔류 블록과의 동일-레이어 충돌, (2) e에서 A의 크레인 진입, (3) f에서 A의 크레인 퇴출, (4) [e,f) 사이 진입하는 잔류 블록을 A가 막지 않을 것, (5) (e,f] 사이 퇴출하는 잔류 블록을 A가 막지 않을 것.

기하 판정 `geom::overlapsAreaIP` 는 Shapely `overlaps` (= 교집합 면적 > 0) 의미론을 재현한다: 진성 변 교차 → 겹침, 아니면 사전 계산된 "엄밀 내부 표본점"의 포함 검사, 마지막으로 정점 내부성 안전망. 변·정점 접촉은 겹침이 아니다(EPS=10^{-9}). 크레인 규칙(`crane::entryOK/exitOK`)은 수직 전용 하강이다. 움직이는 블록의 레이어 k는 잔류 블록의 **레이어 j ≥ k 전부**와 분리되어야 한다 — 훅이 최종 안착 레벨 위의 모든 층을 쓸고 내려오기 때문이다.

왜 bbox 검사로는 안 되는가. 두 이유가 있고 둘 다 측정되어 있다. 첫째, bbox 검사는 거절 방향으로 불건전하다. 레이어가 서로 물리므로 경계 상자가 겹쳐도 합법인 배치가 존재한다(문제 정의 Fig 7(a)). 실제로 우리 보드의 베이-일자별 **최대 bbox 점유율은 103-105%**인 반면 **진짜 다각형 점유율은 58-65%**다(`solver/src/alns.hpp:1300`). 독립 재계산에서도 P6은 다각형 59-63% 대 bbox 107–110%, P4는 65-69% 대 105-109%였다 (`runs/mipboards/*.sol.json`). bbox 오라클을 썼다면 실제로 사용 중인 용량의 3분의 1가량을 스스로 금지했을 것이다. 둘째, bbox는 **크레인 규칙을 표현할 수 없다.** 크레인 관계는 (k, j ≥ k) 레이어 쌍 위의 비대칭 관계로 동일 레벨만 보는 안착 규칙과 다르다. 납작하게 뭉친 상자 하나로는 "블록 1은 블록 2 위로 들어올릴 수 있으나 그 역은 아니다"(Fig 7(d))를 구분할 수 없다.

bbox는 여전히 쓰되 **게이트로만** 쓴다. 레이어별/합집합 경계 상자가 정확 판정 앞에서 단락을 만들고, 비트그리드 점유 사전필터(`occ.hpp`)와 면적 스크린(`areaRoom`)이 혼잡한 (베이, 일자)를 싸게 건너뛴다. 면적 스크린은 설계상 **속도 휴리스틱이지 판정이 아니며**, 삽입 3단계(완전성 그물)에는 걸지 않는다 — 걸었을 때 조밀 보드가 +308% 무너진 이력이 있다.

오라클은 공식 체커에 대해 3중 감사로 검증했다: 균등 표본 0/2,000 불일치, 접촉-동률 케이스 0/90,000, 중첩(nesting) 클래스 0/1,500 (`docs/UNDERSTANDING.md` §3). 그리고 출구에서는 체커가 바이트 권위다. 모든 후보 해는 반환 전에 `utils.check_feasibility` 를 통과하며, 체커를 거치지 않는 유일한 경로는 구성상 항상 실행가능한 자명해다(불변식 4).

4. 탐색 구조

구성. 6개의 삽입 순서(EDD, 여유도, 잡음 EDD 등)를 커널 내장 탐욕 배치기로 재구성해 상위 3개의 서로 다른 레이아웃을 시드로 삼는다. 배치기 `repair::insertGreedy` 는 가장 이른 진입일의 최초 적합을 취하며 호출당

3 단계로 동작한다: ① e=R에서 전체 코너 스캔, ② 잔류 블록 퇴출 이벤트를 올라가며 그 블록 주변만 국소 스캔(공간은 누군가 나가야 열리므로), ③ 남은 날의 희소 조밀 스윕(완전성 그물).

ALNS. 10개의 파괴 연산자를 적응적 룰렛(신규 최적 3점 / 개선 1점 / 수용 0.3점, 감쇠 0.85, 60반복 신용 주기)으로 고른다: 0 무작위, 1 최악-지연(잡음), 2 관련성(베이+시간+공간), 3 크레인-bbox 이웃, 4 크레인-정확 차단자 BFS, 5 베이×시간 슬라이스, 6 준최악(상위 30% 풀), 7 병목 지향(정시 창 안에서 면적 부하 정점을 이루는 (베이,일자)의 잔류자 축출), 8 선호 축출 사슬, 9 지연 최소절단 축출.

연산자 8·9는 무작위 제거가 조합해 낼 수 없는 **가격화된 지향 이동**이다. 8은 선호 베이 이동의 이득(w_3 단위)보다 축출 비용이 작게 유지되는 동안만 그 베이를 가장 낮게 평가하는 잔류자부터 비우고 이동자를 정시 진입으로 강제 삽입한다. 9는 그 Z_1 거울상으로, 지연 블록의 정시 창에 대해 **모든 베이**의 후보 위치를 훑어 축출 집합이 가장 싼 구멍을 찾아 정확히 그 집합만 제거한다 (자기 베이로 제한했을 때는 실측 개선이 3-4회에 그쳤다 — 무작위 제거는 같은 런에서 456-1,071회). 둘 다 §1의 무차원 술어로 서식지 게이팅된다.

수용과 포트폴리오. 정확 목적함수에 대한 SA(T_0 = 0.002 × 초기값), 예산의 75% 이후 절반의 페르소나는 언덕오르기 꼬리로 전환한다. 서버가 4코어를 주므로 OpenMP 4스레드가 서로 다른 페르소나로 병렬 어닐링한다: 제거 크기 배수(0.6/1.0/1.5), 시작 온도(×0.5/1/2), 순수 SA 대 언덕오르기 꼬리, 연산자 부분집합(짝수 스레드 `0x1F` = 0-4 착취, 홀수 스레드 `0x3FF` = 축출 포함 전체), 면적 스크린 체제(필터링 대 절대 오거부 없음). 출발점은 최적 시드 / 새 구성 / 2위 / 3위로 **분지를 헤지**한다 — 어느 분지에 떨어지느냐가 인스턴스당 약 10%(최대 80%)를 흔들고, 목적값이 더 좋은 시드가 더 나쁘게 어닐링되는 사례가 관측되었기 때문이다.

인구 경로. 예산이 빡빡한 인스턴스에는 메메틱 외부 루프를 쓴다. 서로 다른 구성들을 각각 한 번의 버스트(`max(20s, sec/8)`)로 경화해 창시자 집단을 만들고, **합의-잠금 교차**(두 엘리트 부모가 배정에 동의하는 블록만 베이를 잠그고 나머지는 어닐링에 맡김)로 자식을 만들어 최악을 교체한다. 무장 조건은 척도 불변 게이트 `sec/n ≤ 1.6 ∧ sec ≥ 150`이다. 이전의 절대 창($150 \le \text{sec} \le 250$)은 한 인스턴스의 블록당 예산이 만든 인공물이었고, 40-인스턴스 스윕에서 항상-포트폴리오 대비 후회를 1.3%p 악화시켰다.

척도 불변 스케줄. 모든 상수를 n에 상대화했다. 냉각 exp(-0.06/n) /반복, 정체 6n 반복 후 재가열(둘 다 n=200에서 기존 0.9997 / 1200에 정확히 환원된다), 소형 제거 $q \in [2, \max(8, 0.03n)]$, 대형 제거 $q \in [0.05n, 0.15n]$, 정체 20n 후 최적해 복귀, LAHC 이력 4n, 여유도 대리항 가중치 1/n. 절대 냉각률을 쓰면 같은 코드가 n이 커질수록 다른 알고리즘이 된다 — 히든 인스턴스의 크기를 모르는 이상 무차원화는 선택이 아니라 요구사항이다.

5. 파괴/수리 위에 얹은 원자적 이동

파괴/수리는 본질적으로 "제거 후 재삽입"이고 이는 **빈 공간을 요구한다.** 그런데 §6이 보이듯 우리 보드에 없는 것이 바로 그 빈 공간이다. 그래서 빈 공간을 요구하지 않는 원자적 이동을 반복 루프 안에 직접 얹었다.

- **교환(swap, 반복의 2%)**: 두 블록의 슬롯(베이, x, y, 진입)을 통째로 맞바꾼다. 포화 패킹의 고전적 이동으로 여유 공간이 0이어도 성립한다. 개시자는 실제 지연 블록으로 제한했다 — 지연이 없는 보드에서 무작위로 교환하면 건강한 레이아웃을 휘저을 뿐이다(P3 +5.3% 실측). 검증은 동일한 vp 오라클, 가격은 동일한 수용 규칙이다.
- **제자리 이동(shift-in-place, 반복의 10%)**: 잔류 블록을 자기 베이·자기 진입일에서 다른 (x,y)로 민다. 일정이 불변이므로 비용은 vp 검사 한 번 더하기 해당 베이 이웃뿐 — 가용한 구조적 이동 중 가장 싸며, 지연 블록 주변을 조각모음해 이후 삽입이 착지할 자리를 만든다.

- **공간 압축(라운드 종료 시 6회 패스)**: 모든 잔류 블록을 자기 행에서 합법적인 가장 왼쪽 x로 민다. 일정은 불변이다. §2에서 보였듯 위치는 목적함수에 들어가지 않으므로 이 연산은 **구성상 점수 중립**이며, 흩어진 여유 면적을 연속 공간으로 바꾸는 일만 한다. 시간 압축(`compactLeft` : 같은 위치에서 진입을 앞당김, Z_1 비후퇴·Z_2/Z_3 불변)과 짝지어 돈다.

제자리 재배향과 타부식 최급강하 교환도 구현·측정했으나 기본 비활성이다. 특히 타부 최급강하를 매 반복으로 돌리면 파괴/수리 엔진이 굶어 +47%~+1359%로 무너졌다 — §7의 처리량 법칙을 가장 극적으로 보인 사례다.

5.1 이질적 수용 — 채택 후 철회, 판정 방법론의 사례

포트폴리오 절반을 자기척도 LAHC로 어닐하는 이질적 수용은 예선 6개 인스턴스 기준(6시드 통합 +6등수점, P3 −3.3% 재현)으로 한 차례 채택되었다가, 결선용 학습 인스턴스 40개의 **짝지은 부호검정**에서 반대로 판정되어(끄는 쪽이 26승 14패, p=0.04, 평균 −2.34%) 철회되었다. LAHC를 전 스레드에 걸면 예선 기준으로도 손해였다(−7등수점).

이 반전을 굳이 본문에 남기는 이유는 방법론 때문이다. 인스턴스 6개 × 시드 3-6개의 평균 비교는 P3급 분산(단일 추첨 ±10%)을 이기지 못하며, 이번 캠페인에서 시드군이 바뀌자 판정이 뒤집힌 사례가 세 번 나왔다. 결선용 학습 인스턴스 공개 이후 모든 판정을 **40개 짝지은 표본의 부호검정**(같은 인스턴스·같은 시드에서 두 구성을 비교)으로 전환했고, 이 기준이 예선 기반 채택 하나를 실제로 뒤집었다. 표본이 커지면 결론이 바뀔 수 있음을 전제한 판정 체계가, 개별 레버보다 순위에 더 기여했다고 판단한다.

5.2 슬라이스 정제 — 정확 하위해결기를 실행 중 배분으로 (채택)

본 개정의 최대 채택이다. §6의 결정적 증명 — 현재 보드는 임의의 20-30블록 재배치에 대해 **정확히 최적**이다 — 은 남은 개선이 "더 큰 이웃 + 정확 수리"에만 존재함을 뜻한다. 이를 TL 안으로 들여온 것이 두 조각이다.

정확 충돌의 C++화 (`Engine::pairConflicts`): 후보 배치 쌍의 충돌 판정(동층 안착 겹침 + 수직 하강 크레인 스윕)을 엔진 기하로 계산한다. 후보 1,336개에서 충돌 104,717쌍을 **0.09초**에 만든다 — 파이썬/shapely 동등물은 30-60초였고, 이 한 단계가 방법 전체의 성패를 갈랐다(연마 이득이 반복 단가에 정비례함을 후보 30→14→8 실험이 보였다: +0.55%→−0.21%→−0.37%).

두 팔 슬라이스 배분 (`myalgorithm.py _slice_refine`): TL의 꼬리 30%를 조각으로 나눠, 각 조각을 직전 조각의 **초당 개선율이 높았던 팔**에 준다 — (a) 전역 최고해에서 재출발하는 포트폴리오 어닐, (b) 지연 블록을 씨앗으로 ~25블록을 풀어 CP-SAT로 고정된 나머지에 대해 **최적** 재배치하는 연마. 고정 비율은 원리적으로 틀린다는 것이 측정으로 나왔기 때문이다(30% 고정: P6 −3.2%지만 P4 +3.0%, P3는 지연이 없어 연마가 발화조차 안 함). 배분을 실행 중 측정에 맡기면 인스턴스가 스스로 결정하고, 보이는 6개 판에 맞춘 상수가 하나도 없다. CP-SAT가 낸 해는 항상 vp 오라클로 재검증하며, 정제 전체가 체커 게이트의 수리 예비 시간을 침범하지 않도록 마감 이전에 멈춘다 (이 보호가 없을 때 P5 한 시드가 게이트 위반 3건을 1.6초 안에 못 고쳐 전체가 `_trivial`로 붕괴하는 것을 실측했다). memetic(개체군) 경로가 도는 판에서는 통째로 건너뛴다 — 그 판들은 정시 보드라 연마할 대상이 없고, 개체군 탐색이 예산을 더 잘 쓴다.

정식 TL·3시드 평균, 동일 엔진 대비: **P4 −10.6%, P5 −3.6%, P6 −3.4%, P3 +1.9%**, 등수점 +30. TL 초과와 게이트 실패는 0건이다. ortools가 없는 환경에서는 연마 팔이 조용히 빠지고 어닐 조각만 남는데, 그 구조만으로도 P3에서 −5.2%를 냈으므로 대비책이 아니라 그 자체로 유효한 구성이다.

결선용 학습 인스턴스 40개(n 150–300, m 2–5, w_1/w_3 4.2–107)가 공개된 뒤 **짝지은 부호검정**으로 재검증했다: 슬라이스 정제를 끄면 40개 중 26개에서 악화되며(중앙값 +1.79%, 평균 +2.14%), 손실은 가장 큰 판들에 집중된다 (n=300 보드에서 +11~+26%). 부분집합 크기·후보 수·시간 몫의 변형(K=60, 후보 40, 몫 0.45)과 응집 부분집합(씨앗의 베이-시간창 이웃)은 모두 짝지은 검정에서 무승부 또는 열세로, 기본값(K=25, 후보 30, 몫 0.30, 무작위 부분집합)이 최종이다 — 응집 부분집합의 열세는 §6의 증명(혼잡 구간은 이미 최적)과 정합한다: 정확 재배치의 가치는 부분집합의 **다양성**에서 나온다.

6. 설계를 이끈 진단

이 절이 본 보고서의 핵심이다. 우리의 모든 구조적 선택은 아래 네 측정에서 연역되었다.

(a) 수리는 실패하지 않는다 — 블록은 배치되되 늦게 배치된다. 계측 빌드로 잰 파괴/수리 실패율은 **정확히 0.0%**다. P4(n=150, m=3, TL 480초)에서 **1,069,449회 반복 중 실패 0회**, P6(n=250, m=3, TL 900초)에서 **1,767,329회 반복 중 실패 0회** (`runs/faild.log`; 이후 모든 계측 런에서도 동일하게 0.0%). 즉 제거된 블록이 자리를 못 찾는 사건은 일어나지 않는다. 지연은 "배치 불가"가 아니라 "최초 실행가능 위치가 나타나는 날이 늦다"는 현상이다. 따라서 완전성 의미에서 수리를 더 강하게 만드는 방향은 원리적으로 이득이 없다.

(b) 면적 완화 하한 — 용량은 구속 제약이 아니다. CP-SAT 누적 제약으로 면적 완화 하한을 계산했다(`tools/z1lb.py`, `tools/z1bound.py`). 각 블록은 릴리스와 처리시간을 유지한 채 형상만 자기 다각형 면적으로 완화되어 베이의 일일 면적 용량과 경쟁한다. 모든 실제 보드가 이 완화를 만족하므로 그 최적값은 Z_1의 유효한 하한이다. 결과는 **P3–P6 네 판 모두 OPTIMAL로 닫히며 Z_1 하한 = 0**이다 (`runs/z1bound.log`, `runs/z1bound56.log`; P6은 551.7초에 종결). 용량만 놓고 보면 어느 판에서든 모든 블록이 정시일 수 있다는 뜻인데, 우리 보드는 P4 196–229일, P6 4,300일 안팎의 지연을 갖는다. 즉 이 하한은 어디서도 구속하지 않는 **공허한 경계**이고 — 뒤에 보이듯 그 완화의 증거 배정 자체가 다각형 기하로 실현되지 않으므로 — 참 하한과 우리 값 사이의 거리는 이 도구로는 알 수 없다. (측정 부기: 인스턴스 파일 내부의 `name` 필드는 파일명과 어긋나 있다 — `prob_5.json`의 내부 이름은 "prob_3"이다. 모든 평가 경로는 파일명을 쓰므로 영향이 없지만, 내부 이름을 찍는 로그를 읽을 때 주의해야 한다.)

(c) 점유율의 두 얼굴. 베이-일자별 최대 점유율은 다각형 기준 **58-65%**, 경계 상자 기준 **103-105%**다 (`alns.hpp:1300`; 100% 초과는 레이어 중첩 때문에 합법이다). 즉 **피크 일자에도 베이 면적의 35-42%가 비어 있다.** 그런데 `tools/frag_diag.py`는 지연 블록의 대기 일수 중 **51.5%(P4) / 53%(P5) / 18.3%(P6)** 에서 "어딘가에 충분한 여유 면적이 있었다"고 보고한다. 면적은 된다고 했고 기하가 안 된다고 한 것이다. 레이어별·전 베이·중첩 인지 전수 조사(`tools/census.py`)가 이를 날카롭게 만든다: 어떤 지연 블록도 자기 릴리스 창 동안 **어느 베이에도** 실행가능한 위치가 없다.

(d) 결론: 지연은 용량 부족이 아니라 기하학적 파편화 실패다. 여유 면적은 존재하지만 다른 블록들의 경계 상자 안쪽에 흩어져 있어 도착하는 블록이 연속된 자리를 찾지 못한다. §5의 세 이동(교환·제자리 이동·공간 압축), 면적 스크린의 통화를 bbox에서 다각형 면적으로 바꾼 결정, 컴팩트 방향 선호(같은 블록의 방향들은 bbox 면적이 중앙값 1.38배, 최대 3.19배 차이 난다)는 전부 이 진단에서 직접 도출되었다. 보조 진단 둘도 같은 방향을 가리킨다: 위치를 고정하면 베이별 CP가 1초 안에 "일정은 이미 최적"임을 증명하고 (지연은 스케줄링 문제가 아니다), 배정이 고정되면 목적이 베이별로 정확히 분해되어 베이-국소 이동이 정당해진다(§1).

7. 공학 방법론

시드는 복권이고 엔진은 분포다. RNG 소비 순서·인덱스 순서·부동소수 반올림 중 하나만 바뀌어도 모든 궤적이 재추첨되며, 인스턴스별 draw 산포는 ±5-8%(한 T600 쌍에서는 최대 25%)다. 따라서 단일 실행으로 채택한 변경은 하나도 없다. 2-시드 스크리닝으로 후보를 거르고 **공식 시간 제한에서 4-5 시드 분포 비교**로 채택을 결정한다. 단일 draw A/B는 엔진이 아니라 복권 티켓을 비교하는 것이다.

격리, 중첩 금지. 실험은 한 번에 한 레버만 바꾼다. 예컨대 계획-시드 후보는 "구성 슬롯만 교체"하도록 재작성해 슬롯 셔플 교란을 분리한 뒤에야 판정했다. 리팩터링의 유일한 유효 동치 검사는 **고정 반복·동일 시드에서의 바이트 동일성**이며, 고정 카나리(1499 / 3690)와 전상태 감사와 함께 모든 커널 빌드에서 배터리로 돌린다.

측정된 법칙: 반복 처리량이 지배한다. vp 오라클의 거절 경로는 선인출된 선형 스캔 위 정수 비교 두 번으로 약 1–2ns, 사실상 공짜다. 비용은 전부 자격 있는 기하 작업(실제로 겹치는 쌍의 정확 판정, 장애물 집합 재구성, 크레인 경로 논리곱)에 몰려 있다. 이를 두 가지 독립적인 시간-키 인덱스로 공격해 루프 방문을 55-75% 줄였고 참조 구현과 바이트 동일했음에도 이득은 ≤3%에 그쳐 둘 다 기각되었다. 반대로 더 싸 보이던 행 팽창 사전필터는 1.19M 대 1.88M 반복으로 오히려 손해였다. 속도는 오직 반복률 표를 통해서만 품질로 환전된다(커널 ×1.12-1.20 → 통치 반복률 ×1.15 → 대형 보드 −0.4…−8.2%).

따라서 값싼 원자적 이동만이 값을 한다. 최난도 우선 수리 순서는 이미 손에 있는 키(bbox 면적 × 체류 길이)만 쓰므로 반복 수를 건드리지 않는다. 반면 best-of-R 수리, 접촉 점수 기반 best-fit 배치, 래스터-NFP 밀착 배치는 반복당 품질을 사면서 반복 수를 더 크게 잃어 기각되었다(생산 보드 +1.5~8.4%, TL60 +2.8%).

기각된 레버는 기제와 함께 기록해 재론을 막는다: 균등 파괴 연산자 선택 (P5 +5.0%, 4/4 draw), 베이-잠금 심층 탐색, SA 동결 가설(장 TL에서도 수용률 0.48-0.63 유지), 균등 대형 파괴, 창 재적재, 정확 창 분기한정(실제 P4 전량 실행에서 개선 재스케줄 0건), 배정층 Benders, 학습 페르소나 함대(평균 손실은 줄지만 결과는 상관된 draw들의 최솟값이라 이득이 소거됨), 인스턴스별 시드 디스패치(결선은 새 히든 인스턴스이므로 원리적으로 무효 — 코드 삭제).

8. 재현

커널. 단일 번역 단위 `solver/src/api.cpp` 가 `geometry/crane/core/occ/repair/alns/knobs` 헤더를 포함한다. 규모는 C++ 약 4,000줄, 파이썬 드라이버 약 1,000줄. 빌드는 `solver/build.sh`:

```
g++ -O3 -std=c++17 -funroll-loops -fopenmp -shared -fPIC \
    -march=x86-64-v3 -ffp-contract=off -flto \
    (python -m pybind11 --includes) src/api.cpp -o sscore<EXT>
```

채점 서버가 python 3.12이므로 반드시 3.12로 빌드해야 하며, `build.sh` 는 버전을 검사해 경고하고 마지막에 import를 검증한다.

실행. `myalgorithm.py` 의 `algorithm(probᵢnfo, timelimit)` 이 진입점이며 **절대 예외를 던지지 않는다**: 커널과 체커 import는 모듈 수준에서 모두 선택적이고, 최악의 경우 순수 파이썬으로 구성상 항상 실행가능한 자명해를 반환한다. 스레드는 `min(4, cpu_count)` . 채점 서버에는 환경 변수가 설정되지 않으므로 모든 노브가 코드 기본값을 취하고 진단 훅은 전부 불활성이다.

결정성. 반복 예산으로 통치되는 라운드에서 궤적은 (시드, 라운드 수)만의 함수다. 노드 간 바이트 동일성을 실증했으며, 빠른 하드웨어는 라운드를 더 많이 담을 뿐이고 최선값은 라운드 수에 대해 단조다. 로컬에서 잰 시드별 결과가 채점기로 이전되는 근거가 이것이다.

하네스. `tools/eval_prelim.sbatch` (평가), `tools/battery.sbatch` (빌드+감사+ 카나리+고정반복 결정성), `tools/preflight.py` / `tools/test_driver.py` (변경 전·후 검증), `tools/z1lb.py` · `z3lb.py` (면적 완화 하한), `tools/frag_diag.py` · `census.py` (파편화 진단).

채점 단위에 대한 정정 — 목적값이 아니라 등수다

이 절은 예선 채점 구조를 다시 읽고 얻은 것으로, 개발 중 자원 배분의 기준을 바꿨다. 결선은 새 히든 인스턴스(8문제)로 치러지므로 아래의 등수 분포 자체는 결선에 이월되지 않는다 — 그래서 최종 목표는 등수 최적화가 아니라 **모든 문제에서의 최저 목적값**으로 유지했고, 이 절의 분석은 "어느 문제의 개선이 등수를 사는가"라는 개발 중 우선순위 판단에만 사용했다. 순위는 목적값이 아니라 **문제별 등수 점수의 합**으로 매겨진다 (1위 408점, 랭크된 팀 408개, 6문제 합산 최대 2,448). P1·P2는 사실상 전 팀이 동일한 값(11,280 / 31,368)에 도달해 만점으로 묶이므로, **가용 가치는 전부 P3–P6의 등수에 있다.**

`tools/rankcheck.py` 는 예선 리더보드(`runs/leaderboard.json`, 공식 보고서 파싱)를 기준 분포로 삼아 목적값 조합을 이 단위로 환산한다. 현재 최고값에서의 **등수 기울기**는 다음과 같다.

문제	현재 등수	1% 개선당 등수	5% 개선당 등수
P3	7	0	5
P4	13	0	**1**
P5	26	2	13
P6	26	**3**	**16**

P4는 5%를 개선해도 1등수에 그친다(그 구간의 팀 밀도가 희박하다). 반대로 P6는 1%당 3등수다. 백분율 격차가 가장 커 보이던 문제가 실제로는 가장 값싼 문제였다는 뜻이며, 이 지표 없이는 계속 그쪽에 자원을 썼을 것이다. 개선 노력은 P5·P6에 집중하는 것이 옳다.

또한 이 단위에서 보면 성과의 크기가 달라진다. 예선 제출은 총점 2,295(27위, P3 87,140 / P4 3,259,259 / P5 11,089,821 / P6 32,104,939)였고, 최종 구성의 **고정 시드 단일 실행**(출하 조건과 동일)은 P3 83,645 / P4 3,185,673 / P5 10,848,100 / P6 28,953,865로 총점 약 2,358 — 대략 14위권에 해당한다. 1위는 2,440이다. 여러 측정 시드의 최솟값을 성과로 집계하는 것은 채점 현실(고정 시드 1회 실행)을 왜곡하므로 금지했다 — 본 보고서의 모든 현황 수치는 단일-실행 기준이다.

한 가지 함정도 함께 기록한다. 등수 단위로 재판정하면 백분율 평균으로는 중립이던 조합이 크게 이겨 보일 수 있다(`shiftP` off + `retN` off가 시드 34-36에서 +13점). 그러나 새 시드 38-40에서는 −9점으로 재현되지 않았다. **단위를 바꾼다고 다중 시드 확인 의무가 면제되지 않는다.**

미해결 사항 (솔직한 기록)

상위 순위권과의 격차는 여전히 상당하며, §6의 진단이 지목한 바로 그 성분에 정확히 몰려 있다. 최종 구성의 고정 시드 단일 실행 기준(예선 인스턴스, 정식 TL) 전 팀 역대최저 대비 P3 +29% / P4 +67% / P5 +41% / P6 +14%다. 인스턴스 가중치(w_1 = P3 17,778 / P4·P5 13,333 / P6 6,667)로 환산하면 방향이 하나로 좁혀진다: P4에서 상위권은 우리 지연(약 160-176일)의 3분의 2 이하를 유지하고, 후회로 메우는 경로는 산술적으로 불가능하다(P4 격차를 Z_3로 환산하면 우리가 가진 후회 총량을 넘는다). 즉 **잔여 격차는 전부 Z_1에서 나와야 한다.**

같은 진단을 시간 차원에서도 닫았다: 보드의 기하(베이·방향·위치)를 고정하고 전 블록의 진입 시각을 CP-SAT로 전역 정확 재해결하면(`tools/timeopt.py`, 안착 겹침 → 체류 분리, 하강 통로 차단 → 진입·퇴장 회피의 디스정션 모델) P4 766→763일 수준으로 **현재 시각 배치가 사실상 최적**이다. 공간은 국소 최적(§6), 시간은 전역 최적 — 잔여 격차는 오직 **전역적으로 다른 기하 배치**에만 존재하며, 이것이 우리가 도달하지 못한 유일한 것이다.

결선 관련 검증 현황: 첫 결선 제출은 히든 8문제 전부에서 실행 가능 판정을 받았고(TL 초과·붕괴 없음), 결선용 학습 인스턴스 40개(n≤300, m≤5, w_1/w_3 4–107)와 n=300×1500초 장TL 구역까지 실패 0건으로 검증했다. 장TL에서는 예산 증가가 계속 이득으로 전환된다(T600→T1500에서 10판 중 8판 개선, 최대 −12.6%).

하한에 대한 정정

이전 판본은 "면적 완화가 Z_1=0을 허용하므로 잔여 격차 전부가 기하학적 실현 가능성의 문제"라고 적었다. 이 진술의 전제를 직접 시험한 결과 **하한 0은 느슨한 값**임이 드러났다. `tools/planchase.py`로 면적 완화가 내놓은 Z_1=0 계획(`runs/plan_p4.json`)의 베이 배정을 `set_assignment_lock`으로 고정하고 재구성하면, 구성이 80번 블록에서 **실패한다**. 완화의 증거 배정 자체가 폴리곤 기하로는 실현되지 않는다. 따라서 "모든 블록이 정시 가능"은 완화 안에서만 참이며, 도달 가능한 목표로 인용해서는 안 된다. 참 하한은 알려져 있지 않다.

국소 재배치 축은 측정으로 닫혔다

채점 환경에 있는 ortools(9.15.6755)를 이용해, 판의 대부분을 고정하고 가장 혼잡한 (베이, 기간) 구간의 블록 전부를 풀어 **CP-SAT로 정확히 최적화**했다 (`tools/cpsat_lns.py`). 모델은 블록당 후보 하나를 고르는 배타 제약과, 안착 충돌 및 수직 하강 크레인 스윕에서 유도한 쌍별 충돌 절로 구성되며, 반환된 해는 엔진의 vp 오라클로 재검증한다.

이웃	후보/블록	결과	격차
K=15 (지각 최인접)	~35	OPTIMAL 0.2–0.4초	0.0%
K=30 (베이-기간 전체)	~84	OPTIMAL 6.8초	0.0%
K=20, **비코너 격자**(stride 3)	~51	OPTIMAL 0.7초	0.0%

세 경우 모두 vp 위반 0으로 모델의 건전성이 확인되었고, 모델은 Z_2를 빼고 풀어 **대안에 유리한** 조건이었음에도 현재 해를 이기지 못했다. 즉 우리 보드는 가장 혼잡한 20-30블록 이웃에 대해, 코너 후보와 임의 격자 후보 양쪽에서, 모든 베이·방향·진입일에 걸쳐 **정확히 최적**이다. 탐색도 후보 생성기도 병목이 아니다. 잔여 격차는 국소 재배치가 아니라 전역 구조에 있다.

정확 하위해결기를 연마로 쓰기까지 — 단가가 부호를 결정했다

같은 CP-SAT를 혼잡 구간이 아니라 **무작위 부분집합**(지각 블록 하나를 씨앗으로 K개)에 반복 적용하면 이야기가 달라진다. 국소 탐색이 손대지 못하는 판이 실제로 개선된다 — 8월 1일자 보드에서 P6 −1.52%(K=15, 500회, 460회째에도 계속 개선), K=25에서는 337회에 −2.57%. 위반은 매 실행 0건이었다.

문제는 **단가**다. TL 안에서는 그 초를 어닐에서 빼와야 하므로, 판정은 "개선하느냐"가 아니라 "같은 초를 어디에 쓰는 게 더 버느냐"가 된다. 동일 예산·동일 시작으로 겨룬 결과(P6 900초, 전량 어닐 대비):

연마 설정	반복 수	결과
후보 30, 비율 0.35	207	+0.55%
후보 14, 비율 0.35	740	−0.11%
후보 14, 비율 0.22	—	−0.21%
후보 8, 비율 0.22	**985**	**−0.37%** (best rank-1 +16%→+15%)

부호가 오직 반복 단가로 결정된다. 후보 수를 줄여 반복을 5배로 늘리자 손해가 이득으로 뒤집혔고, 더 싸게 만들수록 더 벌었다. 다만 이득은 판을 가린다 — P5 +0.19%, P4 +2.74%로, 정시 구조 연산자와 **동일한 판별 기준**(지각 블록이 많을수록 이득: P6 147개 / P5 66개 / P4 25개)을 보인다.

이 시점의 판정은 "작동하나 단가가 안 맞는다"였다. 반복당 비용의 대부분이 파이썬/shapely 충돌 구축이고 CP-SAT 풀이는 0.2-0.7초에 불과했으므로, 충돌 구축을 C++로 옮기는 것이 유일한 결정적 변경이라고 결론지었고 — **실제로 그렇게 했다.** `Engine::pairConflicts`가 그 변경이며(104,717쌍을 0.09초, 파이썬 대비 수백 배), 그 직후 같은 실험이 P6 500회를 214초에 −4.06%로 뒤집혀 §5.2의 채택으로 이어졌다. 위 표는 "반복 단가가 부호를 결정한다"는 인과를 보여준 중간 기록으로 남긴다.

이웃을 K=48로 키우려면 쌍별 절(O(\text{후보}^2)$, K=20–30에서 48-107초)을 셀-일자 용량 제약으로 바꿔야 하는데, 경계상자로 셀을 채우면 현재 해조차 금지되어 INFEASIBLE이 되고(보드가 경계상자 103-105% 대 폴리곤 58-65%로 서로의 상자 안에 맞물려 있기 때문), 셀 중심 표본으로 바꾸면 반대로 너무 관대해져 vp 위반 17건짜리 "−3.15% 개선"을 낸다. 지연 컷 루프는 8라운드에도 수렴하지 않았다(위반 15-25 유지). K를 30 이상으로 올리려면 충돌 모델이 **정확하면서 준2차 이하**여야 하며, 이는 C++ 측 별도 과제로 남는다.

우리는 어닐링 내부에서 "가장 이른 날, 최초 적합"을 이기는 배치 정책을 아직 찾지 못했다. 접촉/best-fit/NFP 계열은 크레인 규칙과 싸우거나, 아끼는 것보다 많은 반복을 소모했다. 가장 유망한 잔여 방향인 다각형 접촉 후보 생성은 밀집 보드에서 −4.9%(P3, 베이당 66.7블록)와 −1.5%(P6)를 기록했으나 여유 있는 P4/P5에서는 +3.2%/+1.2%였다. 밀도 게이트 뒤에 숨기는 것은 가능하나, 그 게이트를 히든 인스턴스에 걸 만큼 충분한 보드 수에서 검증하지 못했다.

또한 포트폴리오의 스레드 간 최종값 산포가 대형 보드에서 약 14%에 달해 연산의 상당 부분이 지는 분지에서 소모된다. 회수를 시도한 소박한 재배분(2단계 분할, 연속 반감)은 모두 실패했다 — 회수에는 어닐 온도를 보존하는 이주 설계가 필요하다는 것까지가 현재 확인된 사실이다.